CONDITIONAL ROADMAP FOR ADVANCED DEPTH

Track 4 — Future Reference / Advanced Gaps

A map of advanced areas to learn **later**, not a new burden. Track 4 is not for current interviews unless a job description clearly demands it.

· Learn-Only-If Labels

LABEL	MEANING
JD asks	Learn only when a job description or client clearly needs it.
Product-company bar	Learn for algorithm-heavy / FAANG-style rounds.
Cloud / platform role	Learn for DevOps / SRE / platform-heavy roles.
Frontend-heavy role	Learn for React / UI-heavy interviews.
Architect path	Learn for principal / architect roles.

· Advanced Missing Areas

AREA	HIGH-LEVEL TOPICS	WHEN TO LEARN
Advanced DSA	DP, graphs, intervals, trie, union-find, topological sort, Dijkstra, deeper backtracking, segment tree (high-level).	Product-company bar
Deep React / Frontend	Advanced hooks, state architecture, React Query, Redux Toolkit, SSR/ Next.js, accessibility, FE testing, performance profiling.	Frontend-heavy role
Deep AWS	VPC, subnets, security groups, IAM detail, ECS/EKS, Lambda architecture, API Gateway, CloudFront, Route53, multi-region DR.	Cloud / platform role
Kubernetes / SRE	Helm, ingress, HPA/VPA, PDB, service mesh, requests/limits, SLO/ SLA, error budgets, incident command.	Cloud / platform role
JVM Internals	GC algorithms, heap/thread dumps, JFR, memory leaks, class loading, JIT, JVM flags, tuning.	JD asks / architect
Advanced Distributed Systems	Consensus, leader election, quorum, vector clocks, distributed locks, backpressure, consistency models, CRDTs (high-level).	Architect path
Deep NoSQL / Search	Cassandra compaction/tombstones/repair, Elasticsearch/OpenSearch, Redis internals, Mongo aggregation.	JD asks
Security / Compliance	mTLS, cert rotation, deeper OAuth flows, PCI, OWASP ASVS, threat modeling, audit logging, secrets rotation.	Security-heavy role
Data Engineering	Spark, Flink, Kafka Streams, CDC/Debezium, lakehouse, batch vs stream, schema registry.	Data / platform role
Architecture Governance	ADRs, standards, review boards, migration roadmaps, platform strategy, cost governance.	Architect path

· Advanced Question Bank — Future Reference

Algorithms

1. DP memoization vs tabulation
2. Graph BFS vs DFS
3. Detect cycle in directed graph
4. Topological sort
5. Dijkstra (high-level)
6. Merge intervals
7. Trie use cases
8. Union-find use cases
9. Coin change
10. Longest common subsequence

Kubernetes

1. What happens when a Pod crashes
2. Deployment vs StatefulSet vs DaemonSet
3. Service discovery in K8s
4. Readiness vs liveness probes
5. Troubleshoot CrashLoopBackOff
6. What causes pod eviction
7. HPA vs VPA
8. Scale a Java app in K8s
9. Manage secrets securely
10. What happens during a rolling deployment
11. Ingress controller
12. Service mesh

AWS

1. EC2 vs ECS vs EKS
2. When to choose Lambda over ECS
3. SQS vs SNS
4. How S3 achieves durability
5. Secure applications in AWS
6. RDS vs DynamoDB
7. AWS Auto Scaling
8. Design a highly available app in AWS
9. ALB vs NLB
10. Troubleshoot high latency in AWS
11. VPC / subnets / security groups
12. IAM least privilege
13. Multi-region DR (RPO/RTO)

> **AZURE, NOT AWS — POSITION DELIBERATELY**
>
> Your production cloud depth is **Azure** (WSI was Azure-only), so a polished "senior AWS" question list is a positioning trap. Answer AWS questions by mapping to the Azure services you have actually run — and say so: ECS/EKS → AKS, Lambda → Functions, SQS/SNS → Service Bus / Event Grid, DynamoDB → Cosmos DB, RDS → Azure SQL — or frame AWS conceptually. Do not imply AWS production experience you do not have (your own Track 3 "What Not to Claim" rule).

JVM / Performance

1. G1 vs ZGC vs Shenandoah — when to choose each
2. What happens during a Full GC
3. Java Memory Model (JMM)
4. How ForkJoinPool works
5. False sharing and its performance impact
6. Profile a production JVM
7. Analyze a heap dump
8. Analyze a thread dump
9. GC tuning basics
10. JFR use
11. Memory-leak patterns
12. Thread-pool sizing
13. Connection-pool sizing
14. CPU vs IO bottleneck
15. JIT (high-level)
16. Classloader issues

Security

1. mTLS
2. OAuth authorization-code flow
3. PKCE
4. Token rotation
5. Secrets rotation
6. Threat modeling

7 PCI basics
8 Audit logging
9 OWASP top 10
10 API-abuse prevention

· Future Learning Order

- [ ] Deepen **React** only if the next role is frontend-heavy.
- [ ] Add **DP / graphs** only if target clients ask LeetCode-style questions.
- [ ] Add **AWS / K8s / SRE** if the role is cloud / platform-heavy.
- [ ] Add **JVM internals** if performance / platform questions appear.
- [ ] Add **Cassandra / Search** if the JD names them.
- [ ] Add **architecture governance** for the architect / principal path.

> **KEEP THIS IN ITS LANE**
>
> Nothing here is urgent for a normal senior Java contract round. Pick an area up only when a specific JD or client makes it pay off — otherwise your time belongs to Tracks 1 and 2.